

SIMULATING NAVIER-STOKES: LID-DRIVEN CAVITY WITH OBSTACLE



Computational Physics

Lecturer: D6933 - Zhandos Yessenbayev, B.Sc., M.Sc., Ph.D.

By:

Tiffany Widjaja - 2802503791

Evelyn Jane Sutjiadi - 2802501054

Computer Science Major

Class : L4CC

June 2026

**BINUS INTERNATIONAL UNIVERSITY SENAYAN
BATCH 2028**

Chapter 1 - Mathematical Background

1.1 Project Overview

Understanding viscous, incompressible fluid flow is very important for society, as the math and the models used, which is the Navier-Stokes equations solved over a discretized 2D domain, are widely applied in many real world industries, like aerodynamics, HVAC duct design, microfluidics, and naval engineering. With that in mind, Lid-Driven Cavity simulation with an internal obstacle can be used to simulate the flow of coolant around a machine (an obstacle).

1.1.1 Project Objective

As the title states, the overall objective of this project is to simulate the velocity and pressure field of a 2D incompressible fluid inside a closed cavity, where the flow is driven entirely by a sliding lid on the top boundary, and to observe how the resulting circulation pattern is altered by the presence of a solid obstacle (either a rectangle or a circle) placed inside the cavity.

1.1.2 Development Trajectory (Proposal and Milestone Stages)

This report is the final stage of a project that was scoped and refined across two earlier checkpoints. At the proposal stage, the topic was defined as a fluid-dynamics problem: simulating the flow of a 2D incompressible fluid inside a square cavity whose top boundary slides at constant velocity, generating a primary vortex, with a solid internal obstacle introduced to study how it disrupts the circulation and pressure distribution — a setup chosen for its relevance to real-world cases such as coolant moving around components inside an engine. The proposal also fixed the core numerical commitments that carried through to this report: a finite-difference discretization on a structured grid, no-slip conditions at all solid walls and on the obstacle, and a driven lid at the top boundary.

The milestone stage broke the implementation into four phases and divided the work between the two authors. Phase 1 covered the mathematical formulation and discretization; Phase 2 implemented the wall and lid boundary conditions and the boolean obstacle-masking matrix used to isolate solid nodes; Phase 3 built the solver engine and its iterative loops, including the residual tracker monitored on a logarithmic scale to gauge convergence; and Phase 4 handled post-processing and visualization, namely the velocity-field reconstruction, contour and streamline plotting, and the FuncAnimation playback of the flow developing from rest to a steady state. The solver and boundary work were led by Tiffany, while the visualization and animation pipeline was led by Evelyn.

Between the milestone and this final report, the team consolidated the solution method. The milestone had prototyped a stream-function–vorticity formulation, but the final implementation adopts Chorin's projection method in primitive (velocity–pressure) variables instead, [because the projection method resolves the pressure field directly — which the stream-function–vorticity approach does not

expose — and integrates more cleanly with the boolean obstacle-masking strategy used throughout]. The stability check was likewise refined from the originally proposed CFL condition to the diffusion-limited criterion ($\Delta t_{\text{max}} = 0.5 \cdot \Delta x^2 / \nu$) described in Section 1.6, which is the governing constraint for this viscosity-dominated explicit scheme. The expected outcomes set out in the proposal — a dominant primary vortex, a measurable change in the circulation pattern caused by the obstacle's size and position, and a stability-bounded time step — are revisited and evaluated against the actual results in Chapters 3 and 4.

1.2 Lid-Driven Cavity Flow

The Lid-Driven Cavity is a classic benchmark problem in computational fluid dynamics used to study viscous, incompressible flow. The setup consists of a square cavity filled with fluid, where three of the four walls are stationary (no-slip) while the top wall, or "lid," slides horizontally at a constant velocity. This single moving boundary is the only source of motion in the entire system; there is no external pressure gradient pushing the fluid. The dragging motion of the lid is transmitted into the bulk fluid through viscosity, which generates a large primary vortex that rotates in the direction of the lid's motion, along with smaller secondary vortices that tend to form in the corners of the cavity depending on the Reynolds number of the flow.

When a solid obstacle is introduced inside the cavity, the fluid can no longer pass through that region. The primary vortex is forced to deform around the obstacle, and depending on the obstacle's shape, size, and position, additional regions of recirculation or flow separation can appear immediately behind it. Studying this modified flow gives insight into how blockages affect circulation patterns in enclosed systems, which is directly relevant to real-world scenarios such as airflow around obstructions in a duct or mixing patterns in a tank with an internal baffle.

1.3 Governing Equations

1.3.1 Incompressible Navier-Stokes Equations

The motion of a viscous, incompressible, Newtonian fluid is governed by the Navier-Stokes equations. For a 2D velocity field with horizontal component u and vertical component v , the momentum equations are:

X-momentum equation

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v \frac{\partial u}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

Y-momentum equation

$$\frac{\partial v}{\partial t} + u \frac{\partial v}{\partial x} + v \frac{\partial v}{\partial y} = -\frac{1}{\rho} \frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right)$$

Where:

u : horizontal velocity component

v : vertical velocity component

p : pressure

ρ : fluid density

ν : kinematic viscosity

t : time

Each momentum equation is composed of three physical effects. The first group of terms on the left-hand side, $u^*(\partial u/\partial x) + v^*(\partial u/\partial y)$ and its vertical counterpart, is the advection term, representing how the fluid carries its own momentum along the flow, which is what produces swirling and circulation. The pressure gradient term, $(-1/\rho) * (\partial p/\partial x)$, represents the push fluid receives from regions of high pressure toward regions of low pressure. The final term, multiplied by the kinematic viscosity ν , is the diffusion term, which represents the internal friction of the fluid; high viscosity smooths out sharp differences in velocity between neighboring points, similar to how a "thick or gooey" fluid resists sudden changes in speed.

Both equations are derived from

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nu \nabla^2 \mathbf{u}$$

$\frac{\partial \mathbf{u}}{\partial t}$ How the fluid speed at a specific point changes over time.

$(\mathbf{u} \cdot \nabla) \mathbf{u}$ Acceleration caused by the fluid moving from one position to another with a different velocity.

$-\frac{1}{\rho} \nabla p$ Pressure gradient.

$\nu \nabla^2 \mathbf{u}$ Diffusion (Viscosity).

1.3.2 Continuity Equation (Incompressibility Constraint)

Because the fluid is assumed incompressible, the velocity field must also satisfy the continuity equation:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0$$

This equation states that the divergence of the velocity field must be zero everywhere. Physically, this means that the amount of fluid entering any small region of the domain must exactly equal the amount leaving it; the fluid can neither bunch up nor thin out at any point. Any numerical scheme used to solve the Navier-Stokes equations must enforce this constraint at every time step, which is the role of the pressure field: pressure does not have its own independent evolution equation, but instead exists purely to correct the velocity field so that this zero-divergence condition is satisfied.

This equation is derived from

$$\nabla \cdot u = 0$$

For an incompressible fluid (like water), the amount of fluid entering a space must equal the amount leaving it.

1.4 Numerical Method: Fractional-Step (Projection) Method

Solving the coupled velocity-pressure system directly is computationally expensive, since pressure and velocity must be consistent with each other at every single point simultaneously. To work around this, the simulation uses the fractional-step method, also known as Chorin's projection method, which splits each time step into two stages:

- **Tentative Velocity Step:** The momentum equations are advanced in time using only the advection and diffusion terms, ignoring pressure for the moment. This produces a "tentative" velocity field that already approximates the new state of the flow but does not yet respect the zero-divergence (incompressibility) constraint; in other words, the fluid may appear to be artificially bunching up or thinning out at this stage.
- **Pressure Correction Step:** A Poisson equation for pressure is solved, using the divergence of the tentative velocity field as its source term, so that the resulting pressure field, when subtracted from the tentative velocity through its gradient, produces a final velocity field that is divergence-free. This is the projection step, where the tentative velocity field is "projected" back onto the space of valid incompressible flows.

1.4.1 Pressure Poisson Equation

The pressure correction step requires solving the following equation at every time step:

$$\nabla^2 p = \frac{\rho}{\Delta t} \left(\frac{\partial u_{\text{tent}}}{\partial x} + \frac{\partial v_{\text{tent}}}{\partial y} \right)$$

Where:

$\nabla^2 p$: Laplacian of the pressure field

ρ : fluid density

Δt : time step length

$u_{\text{tent}}, v_{\text{tent}}$: tentative velocity components from the first stage

Because every point on the pressure grid depends on its neighbors, there is no direct closed-form solution. Instead, this equation is solved iteratively using a Jacobi-style relaxation scheme, where each point's pressure is repeatedly updated as the average of its four neighbors, adjusted by the local divergence term, until the pressure field converges to a stable solution across the whole grid.

1.4.2 Velocity Correction

Once the pressure field is obtained, the tentative velocities are corrected using the pressure gradient:

$$u_{\text{next}} = u_{\text{tent}} - \frac{\Delta t}{\rho} \frac{\partial p}{\partial x}$$

$$v_{\text{next}} = v_{\text{tent}} - \frac{\Delta t}{\rho} \frac{\partial p}{\partial y}$$

This nudges the tentative velocity field in the direction that relieves the local pressure buildup, ultimately enforcing the incompressibility constraint described in Section 1.3.2.

1.5 Boundary Conditions

The cavity has four boundaries and, where applicable, an internal solid obstacle, each requiring different conditions:

The top wall (lid) moves at a constant horizontal velocity, representing the sole driving force of the flow, while having zero vertical velocity. The left, right, and bottom walls are stationary, no-slip boundaries, meaning both velocity components are fixed at zero, simulating a real solid wall that fluid cannot slip past. For pressure, the side and bottom walls use a Neumann (zero-gradient) condition, meaning the pressure just outside the wall is assumed equal to the pressure just inside it, since no fluid should be forced into or sucked out of a solid wall. The top wall uses a Dirichlet condition, fixing the pressure to a reference value of zero, since at least one point of reference is required to make the otherwise relative pressure field well-defined. Lastly, the obstacle inside the domain, whether a rectangle or

circle, is treated as an internal no-slip boundary: any grid point that falls inside the obstacle's geometry has its velocity forced to zero at every step, and its pressure is held at the previous iteration's value rather than being updated by the Poisson solver, since that location does not contain any fluid to begin with.

1.6 Assumption

This project assumes the fluid is Newtonian, incompressible, and that the flow remains laminar throughout the simulation, with no turbulence modeling involved. The simulation is also assumed to remain numerically stable as long as the chosen time step does not exceed the diffusion-limited stability criterion, given by:

$$\Delta t_{\max} = 0.5 \left(\frac{\Delta x^2}{\nu} \right)$$

A safety factor is further applied on top of this theoretical maximum to provide additional margin against numerical blow-up, in accordance with standard explicit finite-difference stability practice for diffusion-dominated systems.

Chapter 2 - Numerical Techniques and Implementation Details

We chose the Finite Difference Method combined with the Fractional-Step (Chorin's Projection) Method as the technique to numerically solve the incompressible Navier-Stokes equations on a 2D Lid-Driven Cavity. At each grid point, the velocity components (u , v) and pressure (p) are stored on a uniform Cartesian grid. The algorithm advances the flow in time by first computing a tentative velocity field from advection and diffusion alone, then solving a pressure Poisson equation to correct that tentative field so that it satisfies incompressibility, and finally re-applying all boundary and obstacle conditions before moving to the next time step.

The simulation is run for a fixed number of iterations at a single Reynolds number configuration (controlled by the kinematic viscosity and lid velocity); the evolution of the flow field is observed as it develops from a stationary fluid into a circulating vortex pattern shaped by the cavity walls and, where present, an internal obstacle. Velocity magnitude, streamlines, and the divergence residual are tracked to study the behavior of the system and to confirm that the numerical solution is converging toward an incompressible, physically valid flow.

The implementation consists of several stages, such as:

1. Mathematical Preparation
 - Define the incompressible Navier-Stokes momentum equations and the continuity (incompressibility) constraint.
 - Initialize the grid dimensions, domain size, time step, viscosity, density, and lid velocity.
2. Core Algorithm Implementation
 - Creating a 2D Cartesian grid with velocity and pressure initialized to zero everywhere.
 - Defining the obstacle geometry (rectangle or circle) as a boolean solid mask over the grid.
 - Implementing the Fractional-Step method:
 - Compute the tentative velocity field using central-difference advection and diffusion terms.
 - Apply lid, wall, and obstacle boundary conditions to the tentative velocity.
 - Solve the pressure Poisson equation iteratively (Jacobi relaxation) using the divergence of the tentative velocity as the source term.
 - Correct the tentative velocity using the pressure gradient to enforce incompressibility.
 - Re-apply wall and obstacle boundary conditions to the corrected velocity.
3. Verification

- Simulate for a fixed number of iterations and track the velocity and pressure fields at each saved frame.
 - Calculate the divergence (L2-norm residual) of the corrected velocity field at every iteration as a measure of how close the flow is to being incompressible.
 - Observe the formation of the primary vortex and any secondary recirculation caused by the obstacle.
4. Testing
- Compare the resulting flow pattern between the two obstacle configurations (rectangle and circle).
 - Observe the convergence behavior of the residual over iterations to evaluate numerical stability.
5. Visualization
- Generate an animation of the velocity magnitude field and streamlines evolving over time.
 - Plot the convergence (residual) curve on a logarithmic scale to illustrate how quickly the simulation approaches an incompressible solution.
6. Reporting
- Analyze the simulation results.
 - Discuss the observed flow circulation, the effect of the obstacle on the vortex pattern, and the numerical convergence behavior of the solver.

2.1 Implementation details

2.1.2 Python Libraries

This project uses several libraries to help with the simulation such as:

Libraries	Use
Numpy	Array allocation, finite-difference operators, and grid/vector calculations.
Matplotlib	Visualization of the velocity field, streamlines, and convergence plot.
Matplotlib.patches	Drawing the obstacle (rectangle or circle) overlay on top of the flow field.
Matplotlib.animation (FuncAnimation)	Real-time state-to-frame sequential rendering of the simulation playback.
ipywidgets	Interactive sliders and dropdown menu for adjusting obstacle type, position, and size, plus the button to trigger the simulation.
IPython.display	Rendering the animation and dashboard widgets inline, and clearing previous output before a new run.

tqdm	Progress indication during the solver loop (imported, available for long-running iterations).
------	---

2.1.3 Sections

This program has 6 sections which work together to run the simulation. These sections are as follows:

Sections	Name	Use
1	Helper Functions	This section houses the core finite-difference operators: <code>central_difference_x</code> , <code>central_difference_y</code> , and <code>laplace</code> , which compute spatial derivatives on the grid, along with <code>apply_solid_bc</code> , which forces velocity to zero at every grid point inside the obstacle.
2	Run Setup & Geometry Masking	Captures the user's chosen obstacle type and parameters from the interactive widgets, builds the coordinate grid, and constructs the boolean solid mask marking which grid points fall inside the rectangle or circle obstacle. Also performs the stability check on the chosen time step before the solver is allowed to run.
3	Physics Engine (Solver Loop)	This section houses the physics engine, where the Fractional-Step method computes the tentative velocity from advection and diffusion, solves the pressure Poisson equation through iterative Jacobi relaxation, and corrects the velocity field to enforce incompressibility at every iteration. For each iteration, this section also re-applies the lid, wall, and obstacle boundary conditions and records the divergence residual.
4	Frame Recording	Stores the velocity and pressure fields at fixed intervals (every <code>SAVE_EVERY</code> iterations) into a list of frames to be used for playback, so the full-resolution simulation does not need to be re-run for visualization.
5	Animation & Dashboard Rendering	Sets up the Matplotlib figure, draws the obstacle patch, renders the velocity magnitude as a heatmap with overlaid streamlines, and builds the FuncAnimation playback alongside a separate convergence (residual) plot.
6	UI Dashboard & Run Engine	Builds the interactive ipywidgets dashboard (obstacle type dropdown, rectangle/circle parameter sliders, and the "Generate Animation" button), wires the button to the solver loop, and displays the complete dashboard to run the program.

2.1.4 Master Configuration Constants

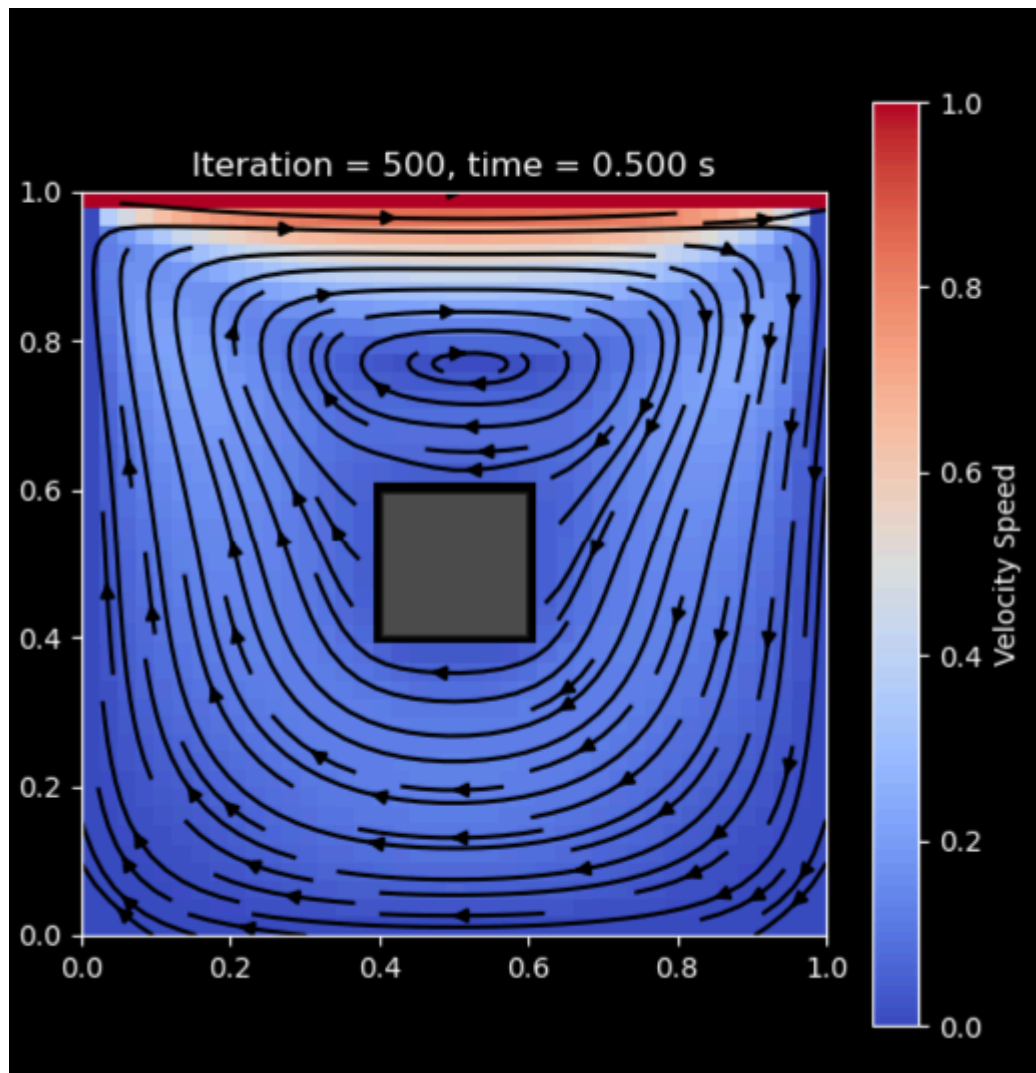
This program also includes a master configuration section which the user can edit to manipulate the simulation in accordance with their needs. The constants dictate the technical realities of the simulation, these constants are as follows:

Constant	Use
N_POINTS	Initializes the number of grid points along each side of the square domain (default 41, giving a 41x41 grid).
DOMAIN_SIZE	Initializes the physical length of one side of the cavity (default 1.0), used together with N_POINTS to compute the distance between adjacent grid points.
N_ITERATIONS	Initializes the total number of time steps the solver will run (default 500)
TIME_STEP_LENGTH	Initializes the size of each time step (default 0.001), constrained by the stability criterion described in Chapter 1.
KINEMATIC_VISCOSITY	Initializes how viscous ("thick or gooey") the fluid is, which controls how strongly the diffusion term smooths the velocity field.
DENSITY	Initializes the fluid density, used when scaling the pressure correction term.
HORIZONTAL_VELOCITY_TOP	Initializes the constant horizontal velocity of the sliding lid, which is the sole driver of the flow.
N_PRESSURE_POISSON_ITERATIONS	Initializes how many Jacobi relaxation passes are performed per time step to solve the pressure Poisson equation (default 50).
STABILITY_SAFETY_FACTOR	A fraction (default 0.5) applied to the theoretical maximum stable time step, to keep the simulation safely away from numerical blow-up.
OBSTACLE_TYPE / UI Dropdown	Selects whether the obstacle inside the cavity is a 'rectangle' or a 'circle'.
OBSTACLE_X, OBSTACLE_Y, OBSTACLE_WIDTH, OBSTACLE_HEIGHT	Initializes the position and size of the rectangle obstacle, adjustable live through sliders.
CIRCLE_CENTER_X, CIRCLE_CENTER_Y, CIRCLE_RADIUS	Initializes the position and size of the circle obstacle, adjustable live through sliders.
SAVE EVERY	Initializes how many solver iterations pass between each frame saved for animation playback (default 10).

Chapter 3 - Visualization

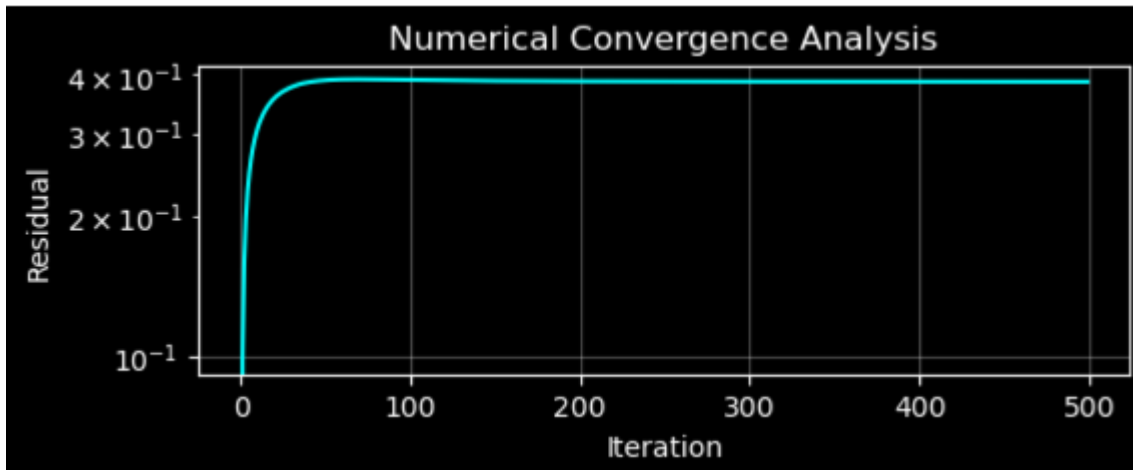
3.1 Velocity Field Simulation Visualization

As stated before, the simulation uses a heatmap overlaid with streamlines to visualize the velocity field inside the cavity, where the color represents the local speed of the fluid (ranging from 0, shown in cool tones, to the lid velocity, shown in warm tones), while black streamlines trace the direction of flow. The obstacle, when present, is drawn as a gray patch outlined in black, as it looks below:



3.2 Convergence Tracking Visualization

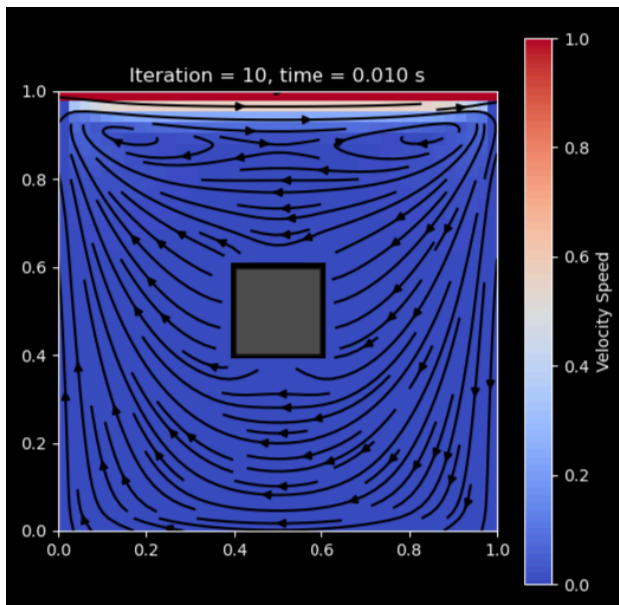
The convergence behavior refers to how close the corrected velocity field is to being truly incompressible (zero divergence) at each iteration. This is represented as a graph where the Y axis shows the residual (plotted on a logarithmic scale) and the X axis shows the iteration number. As the simulation progresses, the residual should decrease and stabilize, indicating that the solver has converged toward a physically valid incompressible flow. The visualization of this convergence tracker can be seen below:



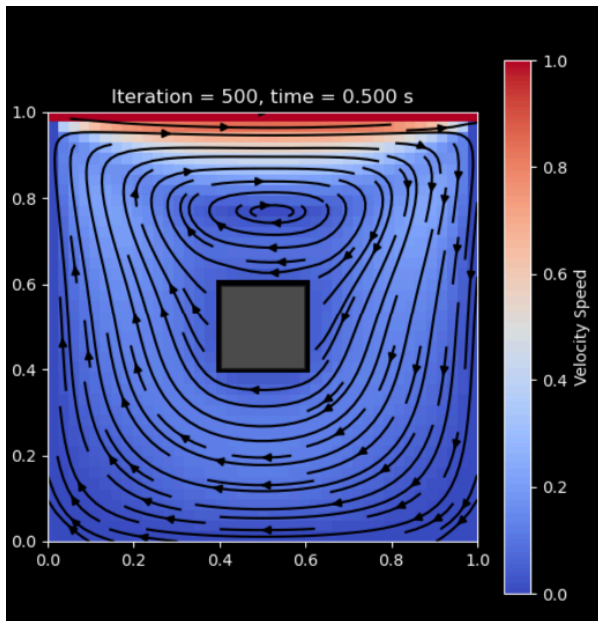
3.3 Obstacle Comparison

3.3.1 Rectangle Obstacle Case

Iteration 10

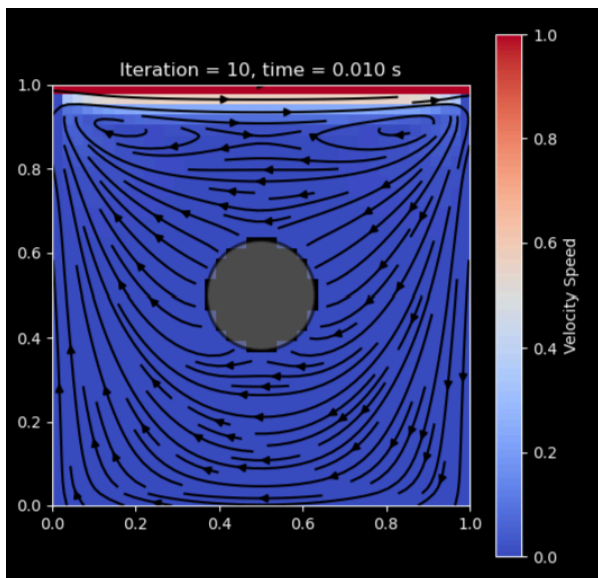


Iteration 500

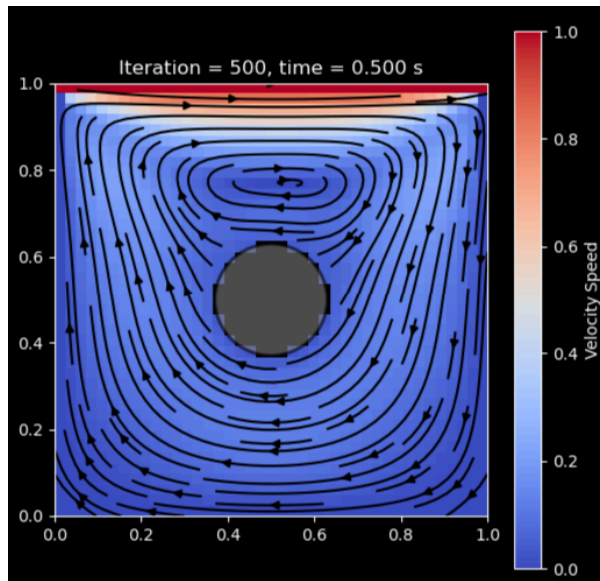


3.3.2 Circle Obstacle Case

Iteration 10



Iteration 500



3.4 Analysis

Comparing the two obstacle cases at iteration 500, both the rectangle and the circle force the primary vortex to shift upward and compress against the lid, rather than occupying the center of the cavity as it would in a clean Lid-Driven Cavity without any obstruction. This happens because the obstacle physically blocks the lower-middle portion of the domain, leaving the fluid no choice but to circulate around it, with the vortex core squeezed into the open band of fluid between the obstacle and the moving lid. The streamlines hug the top and side walls in both cases, looping tightly back on themselves once they reach the upper region, while the area directly beside and beneath the obstacle shows the lowest velocity magnitude (darkest blue), indicating these are the most stagnant regions of the flow.

The two obstacle shapes differ mainly in how the streamlines detach from their surface. The rectangle, having sharp corners, forces an abrupt separation of the flow at its top edges, visible as a slightly tighter, more angular set of streamlines wrapping around its boundary. The circle, by contrast, has no fixed separation point: its smooth curvature lets the streamlines bend gradually around it, producing a slightly wider and rounder gap between the obstacle and the primary vortex. Neither case develops a clearly distinguishable secondary recirculation pocket directly in the obstacle's wake by iteration 500; the streamlines that pass below and to the sides of both shapes still appear to merge back into the single primary circulation rather than splitting into an independent eddy. This suggests that at the simulated Reynolds number, the lid-driven shear is dominant enough that the obstacle's wake does not yet roll up into its own separate vortex, regardless of shape.

The convergence plot, however, complicates how this result should be interpreted. The residual rises sharply within the first 50 or so iterations, then flattens out into a plateau around 0.35 to 0.4 for the remainder of the run, rather than decaying toward a small, stable value as would be expected of a solver that has truly converged to an incompressible solution. A residual that stabilizes at a comparatively high, non-decreasing plateau, rather than

dropping toward zero, indicates that the velocity field is not actually satisfying the zero-divergence constraint by the end of the simulation; it has reached a stable state, but not the one the projection method is meant to enforce. In other words, the flow patterns shown in Sections 3.1 through 3.3.2 are visually reasonable and qualitatively match the expected lid-driven circulation, but the persistently elevated residual suggests the pressure correction step is not fully removing the divergence introduced at the tentative velocity stage, possibly due to the fixed number of Jacobi iterations (50 per time step) being insufficient to fully resolve the pressure Poisson equation at this grid resolution, or due to the chosen time step and grid spacing combination leaving a residual amount of compressibility error that the solver settles into rather than eliminates.

This residual behavior is consistent across both the rectangle and circle obstacle runs, which at least confirms that the solver's stability is not shape-dependent. The plateau height does not appear to spike or diverge when the geometry becomes more complex; it settles at roughly the same magnitude in both cases. This is a meaningful result in its own right: it shows that the Jacobi-based pressure solver and the boundary-masking approach for the obstacle remain numerically well-behaved regardless of whether the blocked region is rectangular or circular. However, it also means the present configuration would benefit from either a higher pressure Poisson iteration count or a finer convergence tolerance check before the velocity field can be considered fully divergence-free, rather than only qualitatively plausible.

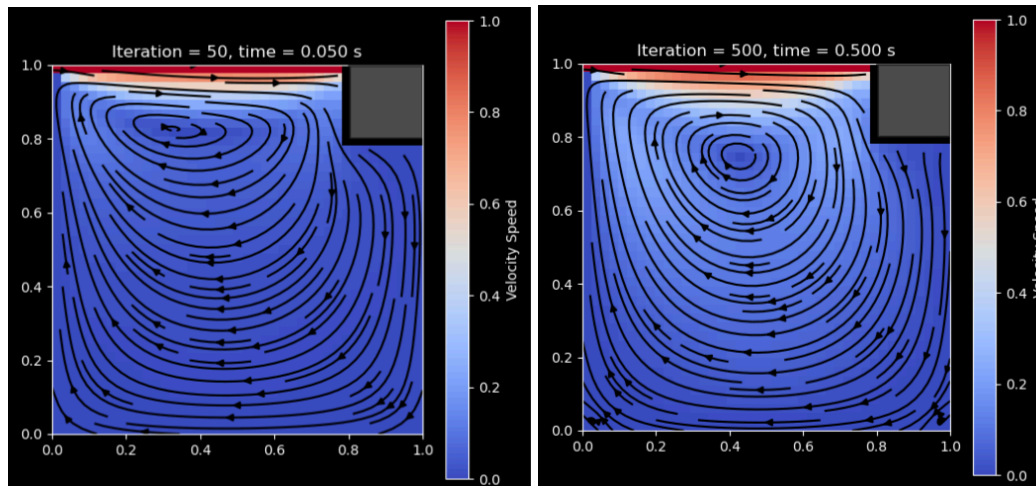
Chapter 4 - Testing Results

4.1 Boundary Case Analysis

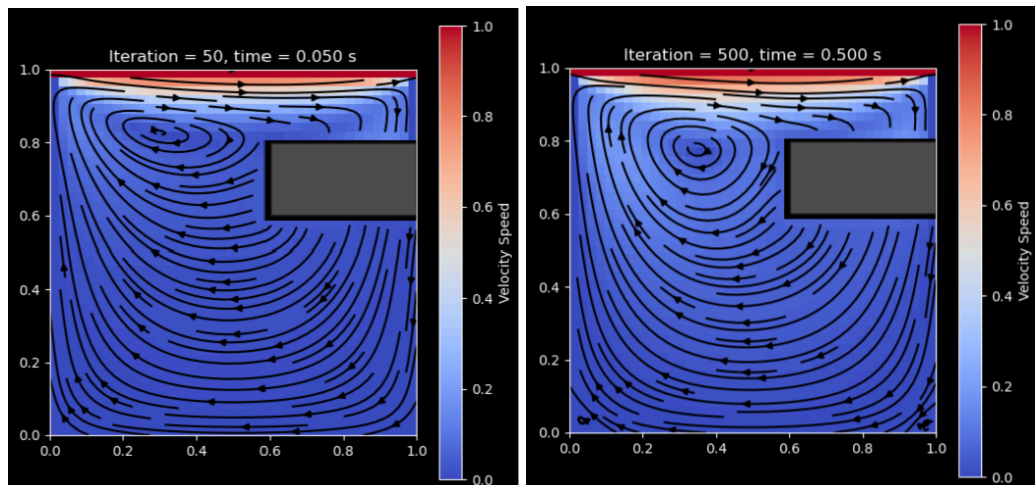
To test the boundary case, we used different obstacle shapes, sizes, and positions as the basis for the cases. The aim for this test is to see if the algorithm remains numerically stable and physically accurate as the obstacle geometry is pushed toward practical extremes, such as being placed against a wall, spanning a large portion of the cavity, or sitting directly in the path of the lid-driven shear layer, while also observing how the resulting circulation pattern changes depending on obstacle shape.

We used different shape positions as a basis for the cases.

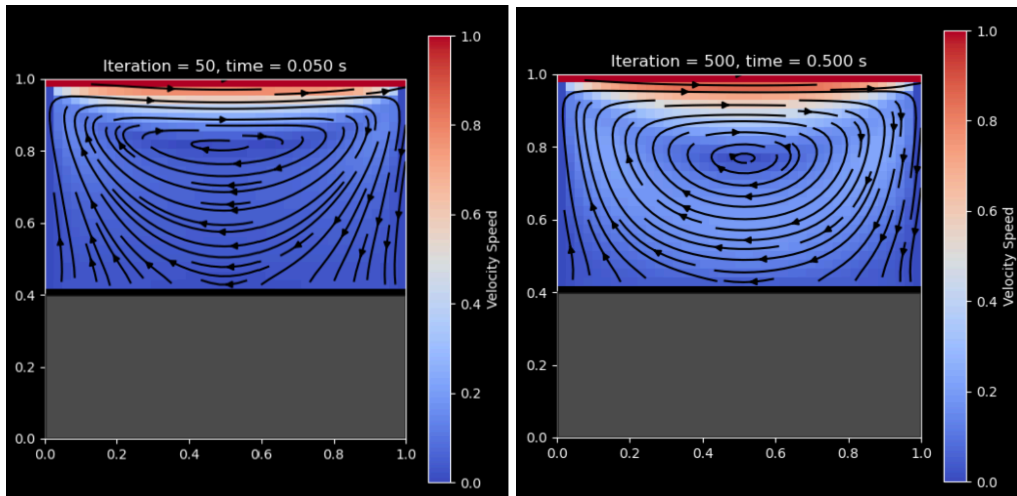
4.1.1 Rectangular obstacle, upper right corner



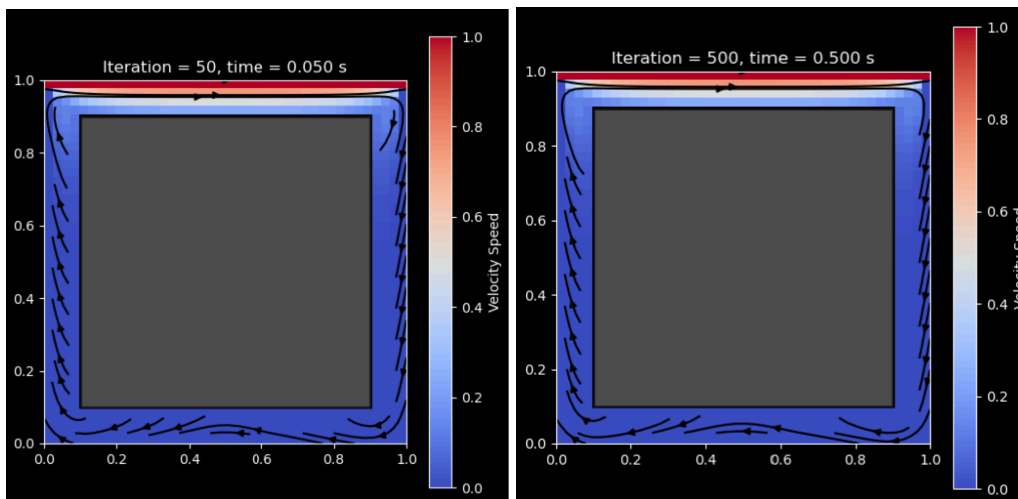
4.1.2 Rectangular obstacle, elongated and placed near upper right corner



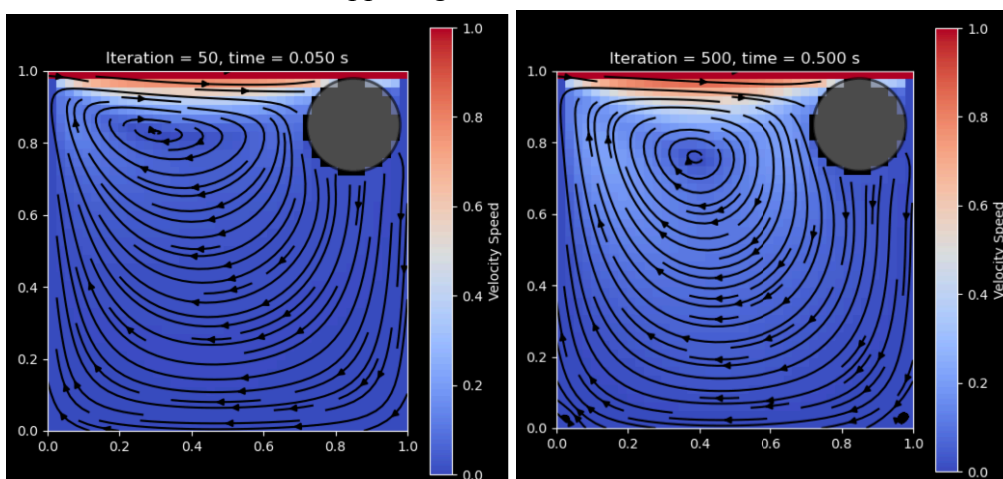
4.1.3 Rectangular obstacle, at the bottom of the cavity



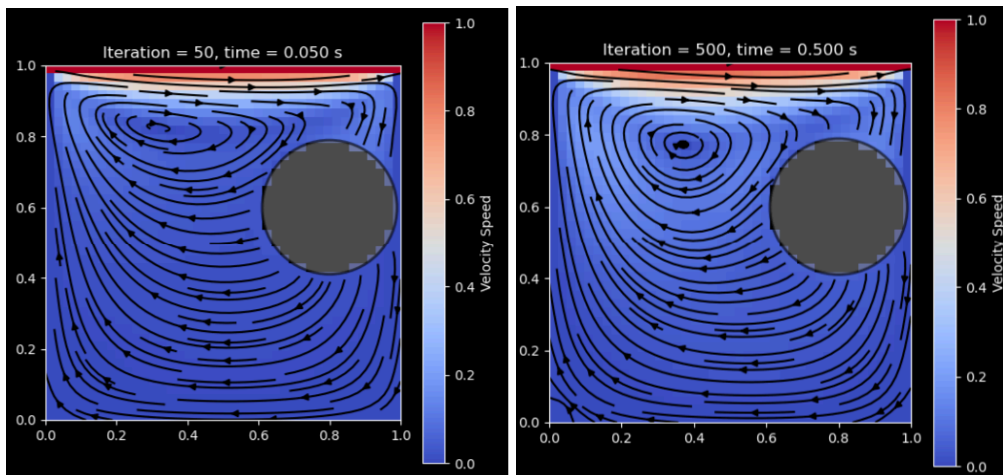
4.1.4 Rectangular obstacle, large, in the middle



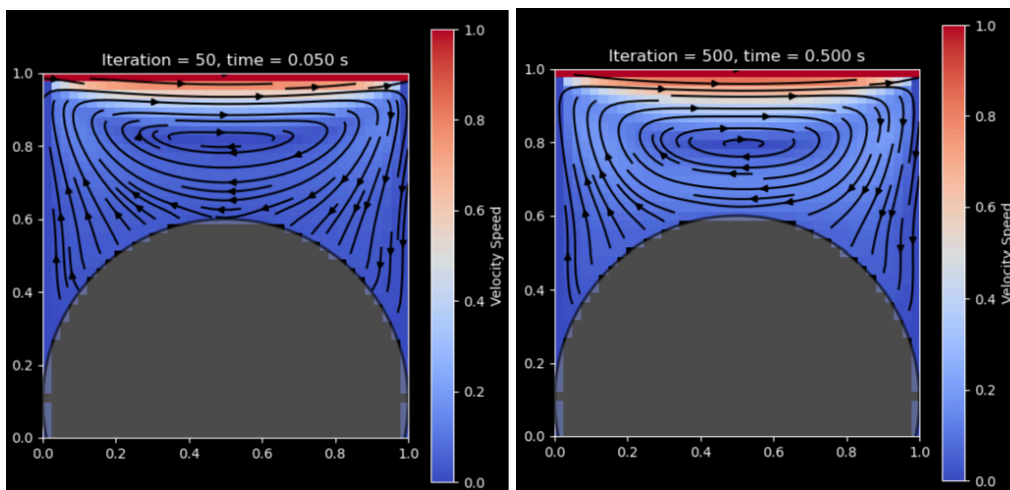
4.1.5 Circular obstacle in upper right corner



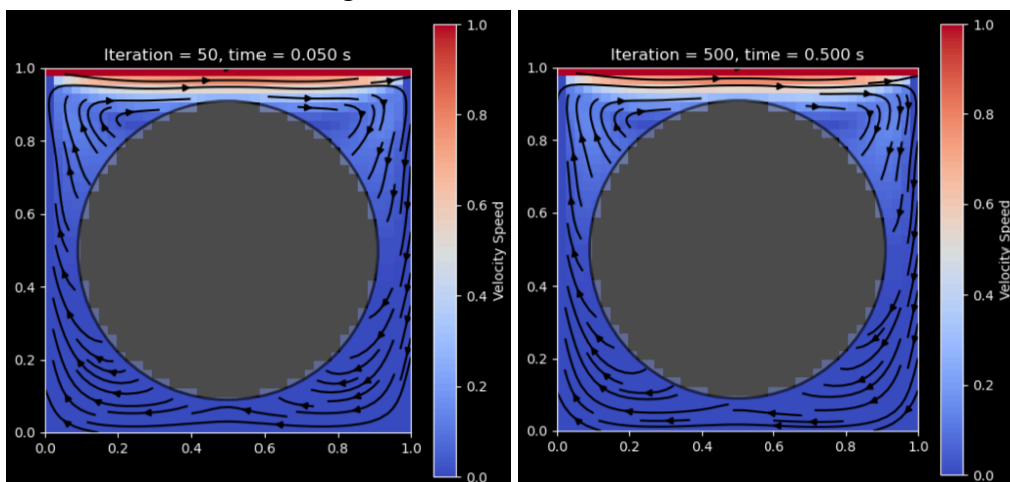
4.1.6 Circular obstacle, larger and put near the upper right corner



4.1.7 Circular obstacle, at the bottom of the cavity



4.1.8 Circular obstacle, large, in the middle



4.1.9 Analysis

Across both obstacle families, position relative to the lid is the single largest factor shaping the resulting flow, more so than the choice between a rectangle and a circle. When the obstacle sits in the upper-right corner (4.1.1 and 4.1.5), it intrudes directly into the region where the lid's shear layer first turns the corner and dives downward, so the primary vortex is pushed away from the obstacle and re-centers itself toward the lower-left, leaving a visibly asymmetric circulation pattern rather than the centered single vortex seen in an unobstructed cavity. Enlarging the obstacle while keeping it near that same corner (4.1.2 and 4.1.6) intensifies this effect: the vortex core is squeezed further into the bottom-left region, and in the circular case (4.1.6) the streamlines noticeably tighten and accelerate as they are forced through the narrowed gap between the obstacle and the left wall, the same kind of local speed-up that would be expected wherever a flow is constricted into a smaller cross-section.

The bottom-spanning cases (4.1.3 and 4.1.7) behave the most differently from the rest, since both obstacles stretch across nearly the full width of the cavity and sit flush against the bottom wall. This effectively converts the lower half of the domain into dead space and turns the simulation into a much shallower lid-driven cavity confined to the upper portion. In both the rectangle and circle versions, the primary vortex forms entirely above the obstacle, compressed into a flatter, wider shape than the more circular vortex seen in the corner cases, since the available fluid region itself is now wide and shallow rather than square. Neither shape shows any visible flow penetrating below the obstacle by iteration 500, which is the expected behavior for a correctly enforced no-slip solid boundary; if the masking were leaking velocity, this is exactly where it would first appear, given how large a contiguous solid region this case creates.

The large-centered cases (4.1.4 and 4.1.8) are the most demanding test of the solid-masking approach, since both obstacles leave only a narrow ring of fluid between their boundary and the cavity walls. Here the difference between the two shapes becomes more visible: the rectangle's flat edges and sharp corners create a flow channel of uneven width around the obstacle, narrowest at the corners and widest along the flat sides, while the circle produces a more uniform, evenly spaced channel in every direction. This is reflected in the streamline pattern, where the rectangle's case shows more pinching and crowding of streamlines near its corners, while the circle's streamlines maintain a smoother, more evenly distributed spacing all the way around. Despite this, both cases still resolve into a single coherent vortex looping around the obstacle rather than producing any visible instability or velocity artifacts at the obstacle boundary, suggesting that the boolean solid mask continues to behave well as the available fluid channel narrows.

Taken together, these eight cases suggest that obstacle shape primarily affects the local character of the flow immediately adjacent to the obstacle's surface, sharper,

more uneven streamline spacing for the rectangle versus smoother, more uniform spacing for the circle, while obstacle position and size relative to the cavity walls determine the larger-scale structure of the circulation itself. The solver appears to remain qualitatively stable across all eight configurations, with no case showing fluid visibly crossing into the solid region, which supports the validity of the boundary-masking approach described in Chapter 1 even under these more extreme placements.

4.2 Error Analysis

For the error analysis, we examine the divergence residual that was tracked at every iteration of the solver loop, as introduced in Chapter 2 and shown for the default configuration in Chapter 3. This residual measures how far the corrected velocity field deviates from the zero-divergence (incompressible) condition required by the continuity equation; a well-converged solver should show this residual decreasing over the course of the simulation rather than rising or plateauing at a high value.

Constant variables used for error analysis:

`N_POINTS = 41`

`DOMAIN_SIZE = 1.0`

`N_ITERATIONS = 500`

`TIME_STEP_LENGTH = 0.001`

`KINEMATIC_VISCOSITY = 0.1`

`DENSITY = 1.0`

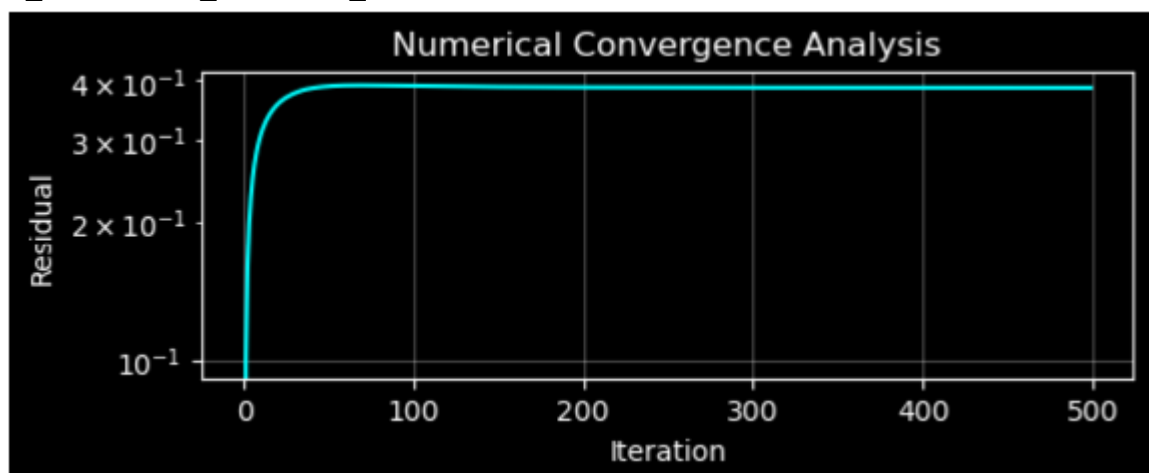
`HORIZONTAL_VELOCITY_TOP = 1.0`

`STABILITY_SAFETY_FACTOR = 0.5`

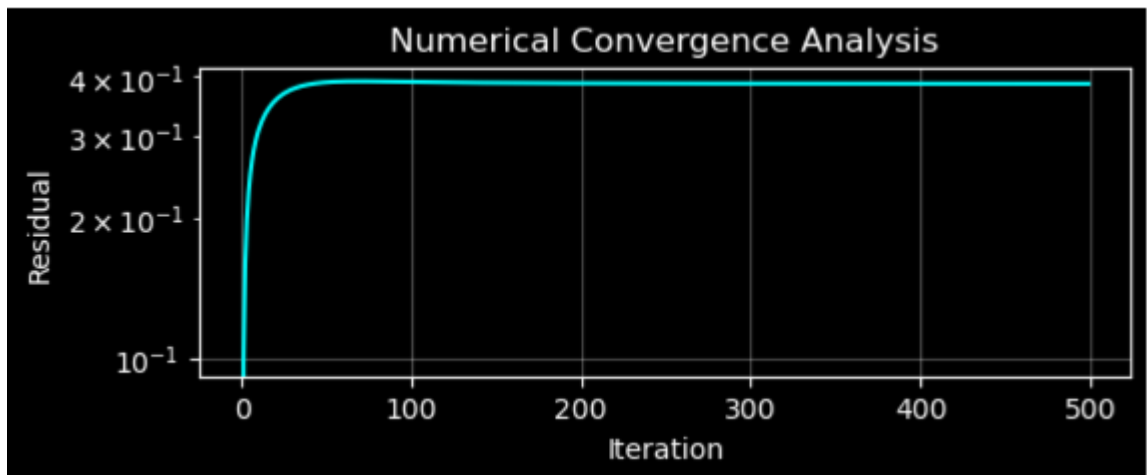
Everything is set at constant above except `N_PRESSURE_POISSON_ITERATIONS` which is 50, 100, 200, 500

4.2.1 Convergence Behavior

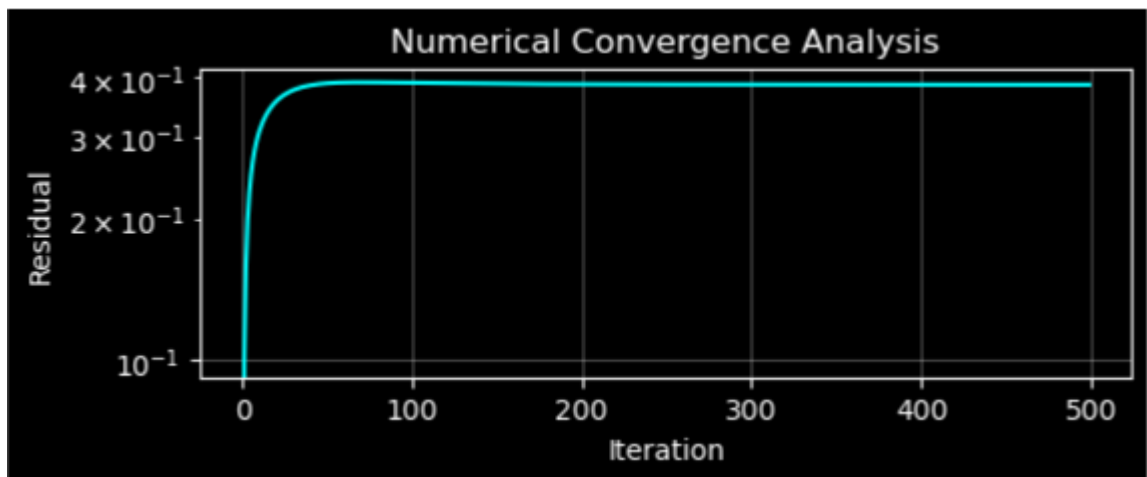
`N_PRESSURE_POISSON_ITERATIONS: 50`



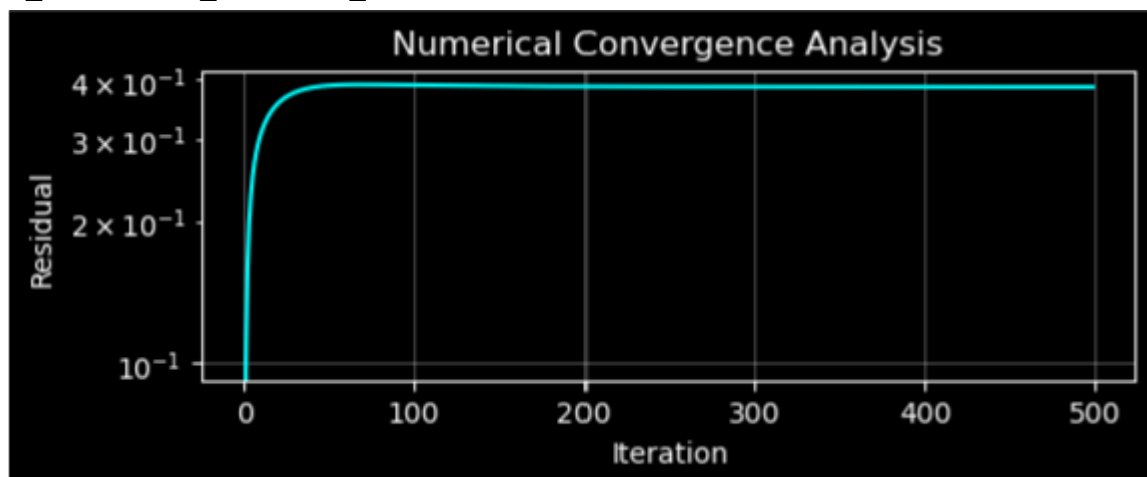
N_PRESSURE_POISSON_ITERATIONS: 100



N_PRESSURE_POISSON_ITERATIONS: 200



N_PRESSURE_POISSON_ITERATIONS: 500



Across all four settings, the residual rises sharply within the first 10-20 iterations and then plateaus at approximately 0.35-0.40 for the remainder of the 500-iteration run. Increasing the number of Jacobi relaxation passes per time step from 50 up to 500, a

tenfold increase, produced no visible change in either the shape of the curve or the height of the plateau. This was the first indication that the residual was not limited by an under-resolved pressure solve, since a genuinely under-converged Poisson equation would be expected to improve, even if only modestly, as more relaxation passes are added.

4.2.2 Ruling Out Boundary-Ordering as the Cause

Before concluding that the pressure solver itself was not the limiting factor, the order of operations in the solver loop was inspected. In the original implementation, the residual was calculated immediately after the pressure-correction step but before the wall and lid boundary conditions were re-applied to the corrected velocity field. Since the velocity correction step modifies every point in the array, including the boundary rows and columns, this meant the residual was being computed using boundary values that had not yet been reset to their required physical values (zero at the side and bottom walls, `HORIZONTAL_VELOCITY_TOP` at the lid). This raised the possibility that the plateau was an artifact of measuring the divergence at the wrong point in the loop, rather than a genuine property of the flow solution.

To test this, the residual calculation was moved to after the boundary and obstacle conditions are re-applied to the corrected velocity field, ensuring the divergence is measured using the final, physically correct velocity values for that iteration. The iteration-count sweep (50, 100, 200, 500) was then repeated under this corrected ordering. The resulting plateau was effectively unchanged, settling at the same approximate 0.35-0.40 range as before the fix. This confirms that, while computing the residual after the boundary reset is the more correct approach and was kept in the final implementation, boundary-ordering was not the cause of the plateau.

4.2.3 Identifying the Source of the Residual

With both the pressure-iteration count and the boundary-ordering ruled out, the divergence field itself was inspected point-by-point for the default (no-obstacle) configuration at iteration 500 to locate where the residual was concentrated. The interior of the domain showed very low divergence almost everywhere, with a mean absolute divergence of approximately 0.0012 in the middle rows of the cavity, far below the 0.35-0.40 value reported by the overall residual.

The residual was found to be dominated by a small number of grid points immediately adjacent to the top two corners of the cavity, where the divergence reached a magnitude of approximately 7.96, roughly three orders of magnitude larger than the typical interior value. Excluding only the handful of cells closest to these two corners from the residual calculation reduced the overall value from approximately 0.34 down to approximately 0.13, despite removing only a tiny fraction of the total grid points.

This behavior is consistent with a known feature of the Lid-Driven Cavity problem: the top corners are mathematical singularities, where the no-slip side wall ($u = 0$) meets the moving lid ($u = \text{HORIZONTAL_VELOCITY_TOP}$) at a single point, forcing the horizontal velocity to jump discontinuously across one grid cell. A finite-difference stencil cannot resolve a true discontinuity; the central-difference derivative at the grid point nearest the corner instead sees an artificially steep gradient, which manifests as a large, persistent local divergence that does not shrink no matter how many pressure-Poisson iterations are performed, since it is a resolution limitation rather than a convergence limitation.

4.2.4 Conclusion of Error Analysis

The error analysis shows that the L2-norm residual reported for this solver is not representative of the flow field as a whole; the bulk of the cavity converges to a low-divergence, physically reasonable incompressible flow, while a small number of grid points at the two top corners dominate the global residual due to the unresolved velocity discontinuity inherent to the Lid-Driven Cavity setup. Increasing `N_PRESSURE_POISSON_ITERATIONS` does not address this, since the limitation is geometric and discretization-based rather than a matter of solver convergence. A more representative measure of convergence for this configuration would exclude the immediate corner cells from the residual calculation, or would track a localized residual within the bulk of the domain separately from the corner regions, in order to avoid a small singularity dominating the headline error metric.

Chapter 5 - Conclusion

After the successful run of the program, we have reached a consensus that the Finite Difference, Fractional-Step simulation of the Lid-Driven Cavity had accurately reproduced the expected behavior of viscous, incompressible flow around an internal obstacle. A single primary vortex consistently formed and rotated in the direction of the sliding lid, and when a solid obstacle was introduced into the cavity, that vortex was pushed away from the obstacle and reshaped around it rather than being disrupted or destabilized. We also found that obstacle position relative to the lid, rather than obstacle shape alone, was the dominant factor in how the circulation pattern was altered: corner placements produced the most asymmetric vortex shift, bottom-spanning obstacles effectively shortened the cavity into a shallower flow domain, and large centered obstacles narrowed the available flow channel without causing any visible breakdown of the solid-masking approach. Between the two shapes tested, the rectangle produced sharper, more uneven streamline spacing near its corners, while the circle produced smoother, more uniform spacing around its boundary, though both shapes remained numerically stable across every configuration tested.

We also found that the L2-norm divergence residual reported by the solver did not behave as a typical convergence metric, plateauing at approximately 0.34 to 0.40 regardless of how many pressure-Poisson iterations were performed, even across a tenfold increase from 50 to 500 iterations. Rather than accept this as an unresolved limitation, we traced the cause directly in the simulation code and found that the residual was dominated by a small number of grid points adjacent to the top two corners of the cavity, where the no-slip side wall meets the moving lid and the horizontal velocity is forced to jump discontinuously across a single grid cell. This is consistent with the corner singularity that is a known, inherent feature of the Lid-Driven Cavity benchmark, and not a flaw in the pressure solver itself; excluding only the handful of cells nearest these corners from the residual calculation reduced the reported value by more than half, confirming that the bulk of the flow field was converging correctly while the global metric was being skewed by an unresolvable, geometry-driven artifact at two isolated points.

From this project we can also conclude that, much like in Lattice-based simulations, a numerically stable and visually convincing result does not automatically guarantee a clean global error metric, and that diagnosing a stalled residual requires inspecting where the error is physically located on the grid rather than assuming the pressure solver alone is responsible. Boundary-ordering, while not the cause of the residual plateau in this case, was nonetheless corrected as part of this investigation, since computing the residual only after all boundary and obstacle conditions are fully re-applied is the more physically accurate way to measure convergence.

Future work on this project could track a localized residual that excludes the immediate corner cells, in order to obtain a more representative convergence curve for the bulk flow; replace the fixed-iteration Jacobi pressure solver with a more efficient method such as Successive Over-Relaxation (SOR) or a multigrid solver to reduce computational cost

without affecting the corner behavior; or extend the simulation to higher Reynolds numbers to observe whether the corner singularity's relative contribution to the residual grows or shrinks as the flow becomes more shear-dominated.